Software Quality Assurance (SQA)
Complete Study Guide
(English Terms + Arabic Explanation)

This guide covers the entire Part 1 material from the uploaded slides, including all key definitions, models, quality factors, comparisons, and exam-focused explanations. It is designed so you can study from this document without needing to return to the original slides.

1. Introduction to Software Quality Assurance (SQA)

- Software Quality Assurance (SQA): A planned and systematic set of activities that provides confidence that software conforms to technical and managerial requirements.
- Arabic: ضمان جودة البرمجيات هو مجموعة إجراءات مخططة ومنظمة للتأكد من أن البرنامج يحقق المتطلبات الفنية والإدارية، ويلتزم بالوقت والميزانية.
- Goals of SQA: Prevent defects, detect defects early, reduce cost of correction, improve customer satisfaction, and ensure reliable and maintainable software.

2. Why Software is Different from Industrial Products

- Product Complexity: Software may have millions of operational modes; industrial products usually have far fewer.
- Product Visibility: Software is invisible, so defects cannot be detected by simple visual inspection.
- Limited Opportunities to Detect Defects: Most defects are detected during development only.

3. Environments for Which SQA Methods Are Developed

- Students and pupils.
- Software amateurs (hobbyists).
- Professionals in other fields (engineering, economics, research).
- Professional software developers (analysts, programmers, testers).

4. Characteristics of the SQA Environment

- Contractual Conditions.
- Customer–Supplier Relationship.
- Requirement for Teamwork.
- Cooperation with Other Development Teams.

- Need for Interfaces with Other Systems.
- Continuation Despite Team Changes.
- Long-Term Maintenance.

5. Basic Terminology

- Software = Programs + Procedures + Documentation + Data.
- Programs: Code that performs required functions.
- Procedures: Instructions and schedules for operation.
- Documentation: Requirements, design documents, user manuals.
- Data: Information required for system operation.

6. What is Quality?

- Quality: The degree of excellence of something.
- IEEE Definition: The degree to which software meets specified requirements and customer expectations.
- Pressman Definition: Conformance to stated requirements, documented standards, and implicit professional expectations.

7. Approaches to Quality

- Transcendental View: Quality is absolute and universally recognizable.
- Product View: Quality can be measured objectively.
- User View: Fitness for use.
- Manufacturing View: Quality results from proper development.
- Value-Based View: Quality depends on cost-benefit trade-offs.

8. Error, Fault, Failure, Defect

- Error: Human mistake.
- Fault (Bug/Defect): Incorrect code or design introduced by an error.
- Failure: Observable incorrect behavior during execution.
- Chain: Human Error → Software Fault → Software Failure.
- Not every Error becomes a Fault, and not every Fault leads to a Failure.

9. Fault/Failure Concepts

- Latent Error: Exists but has not been recognized.
- Dormant Fault: Exists but has not been activated.
- Active Fault: Activated and producing incorrect state.
- Recursive Nature: A failure in one component can become a fault in a larger system.

10. Quality Models Overview

- Quality models transform abstract quality concepts into measurable structures.
- Structure: Factors → Criteria → Metrics.
- Standard Models: McCall, ISO/IEC 9126.
- Specific Models: FURPS, GQM.

11. McCall's Quality Model (11 Factors)

- Product Operation Factors: Correctness, Reliability, Efficiency, Integrity, Usability.
- Product Revision Factors: Maintainability, Flexibility, Testability.
- Product Transition Factors: Portability, Reusability, Interoperability.

12. Definitions of McCall Factors

- Correctness: Degree to which software satisfies requirements.
- Reliability: Probability of failure-free operation.
- Efficiency: Use of resources such as CPU and memory.
- Integrity: Security and protection from unauthorized access.
- Usability: Ease of learning and use.
- Maintainability: Ease of locating and fixing faults.
- Flexibility: Ease of adapting to new requirements.
- Testability: Ease of testing.
- Portability: Ease of moving to another environment.
- Reusability: Ability to use components in new projects.
- Interoperability: Ability to interact with other systems.

13. ISO/IEC 9126 Software Quality Model

- Six Characteristics:
- 1. Functionality
- 2. Reliability
- 3. Usability
- 4. Efficiency
- 5. Maintainability
- 6. Portability

14. ISO/IEC 9126 Sub-characteristics

- Functionality: Suitability, Accuracy, Interoperability, Security.
- Reliability: Maturity, Fault Tolerance, Recoverability.
- Usability: Understandability, Learnability, Operability, Attractiveness.
- Efficiency: Time Behaviour, Resource Utilization.

- Maintainability: Analyzability, Changeability, Stability, Testability.
- Portability: Adaptability, Installability, Co-existence, Replaceability.

15. FURPS Model

- F = Functionality
- U = Usability
- R = Reliability
- P = Performance
- S = Supportability (extend, adapt, correct)

16. GQM (Goal-Question-Metric)

- Step 1: Define Goals.
- Step 2: Derive Questions.
- Step 3: Define Metrics.
- Benefits: Focuses measurement on project goals.
- Disadvantages: Requires additional effort and may be error-prone.

17. Measuring Quality

- User View Measures: Reliability, Availability, Usability.
- Manufacturer View Measures: Defect Counts, Rework Costs.

18. Reliability

- Reliability: Probability that software functions correctly for a specified period under specified conditions.
- R(t): Probability that the system conforms to specifications throughout time t.

19. Availability

- Availability: Probability that the system is operational at any given time.
- A = Percentage of time the system is ready for service.

20. Safety

- Safety: Property that the system will not endanger human life or the environment.
- Safety-Critical Systems: Systems whose failure may cause injury or damage.

21. Maintainability

- Maintainability: Ability to repair and modify software efficiently.

- MTTR (Mean Time To Repair): Average time needed to restore service.

22. Defect Counts and Relative Cost of Correction

- Defect Counts measure the number of identified defects.
- The later a defect is discovered, the more expensive it is to fix.

23. Fault Classification

- Nature: Random/Hardware vs Logical/Systematic/Design faults.
- Duration: Permanent, Transient, Intermittent.
- Extent: Localized, Global.

24. Degradation Faults vs Design Faults

- Degradation Faults: System worked before, then broke (wear-out, damage).
- Design Faults: System never worked correctly because of flawed design.

25. Dealing with Faults

- Fault Prevention: Avoid introducing faults.
- Fault Elimination: Find and remove faults.
- Fault Forecasting: Estimate current and future defects.
- Fault Tolerance: Continue operating despite faults.

26. Main Causes of Software Faults

- 1. Faulty requirements definition.
- 2. Client–developer communication failures.
- 3. Deliberate deviations from requirements.
- 4. Logical design errors.
- 5. Coding errors.
- 6. Non-compliance with standards.
- 7. Testing shortcomings.
- 8. User interface and procedure errors.
- 9. Documentation errors.

27. Cost of Quality

- Includes all costs related to prevention, appraisal, and failures.
- Higher quality reduces long-term cost and maintenance effort.

28. Quality Assurance (QA)

- QA: Planned and systematic activities that provide confidence that quality requirements will be met.
- Focuses on processes and management auditing/reporting.

29. Quality Planning

- Process of identifying product and process quality requirements.
- Produces the Quality Assurance Plan.

30. Quality Assurance Plan

- Defines desired product qualities, quality goals, standards, risk management, and assessment processes.
- Should be short and concise.

31. Typical Contents of an SQA Plan

- Purpose, References, Management, Documentation, Standards, Reviews and Audits, Testing, Problem Reporting, Tools, Code Control, Supplier Control, Records, Training, Risk Management.

32. Quality Control (QC)

- Operational techniques used to fulfill quality requirements.
- Includes inspections, reviews, and testing.
- Objective: Minimize defects and increase product quality.

33. QA vs QC

- QA is process-oriented and preventive.
- QC is product-oriented and detective.

34. Quality Design vs Quality of Conformance

- Quality Design: Quality specified in requirements and design.
- Quality of Conformance: Degree to which implementation follows the design.

35. Quality Assurance System

- Organizational structure, responsibilities, procedures, and resources for implementing quality management.

36. Process and Product Quality

- Product quality is strongly influenced by process quality.
- In software, the relationship is complex and affected by human skill and project context.

37. Practical Process Quality

- Define standards.
- Monitor adherence.
- Report to management and customers.
- Avoid applying inappropriate standards.

38. Most Important Comparisons for the Exam

- Error vs Fault vs Failure.
- Reliability vs Availability.
- QA vs QC vs Quality Planning.
- McCall vs ISO/IEC 9126 vs FURPS.
- Design Faults vs Degradation Faults.
- Quality Design vs Quality of Conformance.

39. Memorization Tips

- McCall = 11 factors (5 Operation, 3 Revision, 3 Transition).
- ISO/IEC 9126 = 6 characteristics (Functionality, Reliability, Usability, Efficiency, Maintainability, Portability).
- FURPS = Functionality, Usability, Reliability, Performance, Supportability.
- GQM = Goal → Question → Metric.

40. Final Study Strategy

- 1. Understand definitions.
- 2. Memorize comparisons.
- 3. Review quality models.
- 4. Solve multiple-choice questions.
- 5. Repeat high-yield concepts.

Final Advice

If you master this guide, especially the definitions, comparisons, and quality models, you will be fully prepared for the SQA Part 1 exam and should not need to return to the original slides.